Optimal TPU v5e Precision Formats

https://gemini.google.com/app/79743c4e55d38433

User prompt: What is the best precision format to run on the TPU v5e? I'm only talking about one TPU. That means with only 16 GB of VRAM. I will be using this through Google Colab. It is currently available on the Colab program.

Response: The best precision format for a single TPU v5e (16 GB HBM) depends on your specific workload: **`bfloat16` (BF16)** is optimal for training and general fine-tuning, while **`int8`** is best for maximizing throughput and parameter fit during inference.

TPU v5e Matrix Multiply Units (MXUs) do not support native hardware acceleration for FP8 (which arrived in TPU v6e/Trillium). Its hardware is specifically built around BF16 and INT8.

Hardware Compute & Memory Comparison

Precision	Peak Compute per Chip	Memory Usage / Parameter	Primary Use Case
`bfloat16`	**197 TFLOPs**	2 bytes	Training, fine-tuning (LoRA/QLoRA), standard inference
`int8`	**393 TOPs** (2× BF16)	1 byte	Inference serving, high-throughput batching, fitting larger models
`float32`	Emulated / downcast	4 bytes	Accumulator registers, loss computation, numerically sensitive heads

1. Training & Fine-Tuning: Use `bfloat16` (Mixed Precision)

For any training or fine-tuning (e.g., via JAX/Flax or PyTorch/XLA):

- **Dynamic range:** `bfloat16` preserves the exact 8-bit exponent width of `float32`, eliminating the underflow/overflow problems and loss scaling gymnastics required by `float16`.
- **Native MXU execution:** On TPUs, the systolic array automatically runs matrix multiplications in `bfloat16` while accumulating results in `float32` registers at full hardware speed.
- **16 GB constraint:** Storing weights, activations, and optimizer states in BF16 allows you to fit small LLMs (~1B–3B parameters full fine-tune, or 7B–8B using LoRA with frozen base weights).

2. Inference & Serving: Use `int8` (or AQT / Quantization)

If you are deploying a model for inference within Colab:

- **2× Compute Throughput:** TPU v5e features dedicated 8-bit integer multipliers, delivering **393 TOPs**—exactly double the compute capability of `bfloat16` (197 TFLOPs).

- **Doubled Parameter Capacity:** In raw 16-bit format, an 8B model requires ~16 GB just for weights, causing Out-Of-Memory (OOM) errors once KV cache and runtime overhead are included. Quantizing weights (and optionally activations via AQT/JAX) to `int8` halves the model footprint to ~8 GB, leaving plenty of the 16 GB HBM headroom for KV caching and larger batch sizes.

Precision Pitfalls to Avoid on TPU v5e

- **Avoid `float16` (FP16):** Unlike NVIDIA GPUs with Tensor Cores optimized for IEEE FP16, TPUs emulate FP16 arithmetic. Running FP16 often leads to compiler-inserted conversion ops and lower throughput than BF16.
- **Avoid `float32` (FP32) for Tensors:** Allocating large tensors in pure FP32 wastes memory bandwidth and capacity. The XLA compiler will either automatically downcast operations to BF16 for MXU multiplication or fall back to slower vector units.
- **Avoid expecting hardware `fp8`:** TPU v5e lacks native FP8 tensor core operations. FP8 routines either trigger software emulation or dequantization to BF16 prior to the systolic array, canceling out expected speedups.

Quick Implementation Setup (Google Colab)

JAX / Flax

```
import jax

# Default all float computations to BF16 to save HBM
jax.config.update("jax_default_matmul_precision", "bfloat16")

import jax.numpy as jnp
# Use jnp.bfloat16 for model layers and weights
dtype = jnp.bfloat16
```

PyTorch / XLA (`torch_xla`)

```
import torch
import torch_xla.core.xla_model as xm

device = xm.xla_device()

# Wrap execution with torch.autocast for XLA
with torch.autocast(device_type="xla", dtype=torch.bfloat16):
    output = model(inputs.to(device))
```

Sources

1. Improve your model's performance with bfloat16 | Cloud TPU
2. TPU v5e - Google Cloud Documentation
3. Google TPU v5 Explained: Architecture, PCB Layout & Deployment

---

User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?

Response: To maximize token generation on a single 16 GB TPU v5e, your optimal format depends strictly on your batch size, because autoregressive token generation shifts between being **memory-bandwidth bound** and **compute bound**.

The Two Regimes of Token Generation

Metric to Maximize	Batch Size (B)	Bottleneck	Winning Precision Format	Why It Wins
Lowest Latency (Fastest stream for 1 user)	$B = 1$	**Memory Bandwidth** (819 GB/s ceiling)	**W4A16** (INT4 Weights, BF16 Activations)	Cuts byte transfers from HBM by 75% compared to BF16.
Maximum Throughput (Total tokens/sec across streams)	$B \geq 4$	**Matrix Unit Compute** (MXU TOPs)	**W8A8** (INT8 Weights, INT8 Activations)	Hits native hardware peak of **393 TOPs** (2× BF16).

1. Single-Stream Generation ($B = 1$): Use W4A16 (Weight-Only INT4)

During single-token decoding at batch size 1, the hardware does almost zero compute per byte loaded—every single layer's weights must be pulled from the 16 GB HBM into the chip just to predict a single token.

The theoretical maximum speed for an 8B parameter model is bounded by the memory bus ($819\ \text{GB/s}$):

$$\text{Theoretical Max Tokens/Sec} \approx \frac{\text{Memory Bandwidth (GB/s)}}{\text{Model Size (GB)}}$$

- **BF16 (~16 GB model):** $\frac{819}{16} \approx \mathbf{51}\ \text{tokens/sec}$
- **INT8 (~8 GB model):** $\frac{819}{8} \approx \mathbf{102}\ \text{tokens/sec}$
- **INT4 (~4 GB model):** $\frac{819}{4} \approx \mathbf{204}\ \text{tokens/sec}$

TPU v5e does not have native 4-bit multipliers, but in **W4A16**, the weights are stored in HBM as 4-bit integers and unpacked into BF16 inside local registers before feeding the MXU. Because memory transfer is the bottleneck at $B = 1$, the time saved reading from HBM dwarfs the tiny cost of unpacking bits in SRAM.

2. Batched Throughput ($B \geq 4$): Use W8A8 (AQT INT8)

When generating tokens for multiple prompts simultaneously, the operational intensity increases, and generation becomes **compute-bound**.

In this regime, W4A16 loses its advantage because the Vector Units spend too much time unpacking 4-bit weights, while the Matrix Units sit idle.

- **Native Hardware Acceleration:** TPU v5e has dedicated 8-bit integer systolic arrays delivering **393 TOPs** (vs. 197 TFLOPs for BF16).
- **Full INT8 Pipeline:** Both weights and dynamic activations are quantized to INT8, meaning matrix multiplications execute directly in hardware at double the speed without intermediate dequantization.
- **Tooling:** On JAX/XLA, this is implemented using **AQT (Accurate Quantized Training & Target)** or Google's **MaxText** inference framework.

3. Don't Ignore the KV Cache

Within 16 GB of HBM, an 8B model in BF16 leaves zero headroom for a multi-token context window. Precision of the KV cache dictates how long your sequences can be before running out of memory or degrading generation speed:

- **Format:** Set **KV Cache to INT8** (`kv_cache_dtype="int8"` in JAX/MaxText or vLLM).
- **Impact:** Halves KV memory bandwidth consumption per decoding step and doubles the usable context length within your 16 GB limit.

Summary Recommendation for Colab

- If running a **single interactive chat loop** (1 prompt at a time): Run **W4A16** (or W8A16 if your framework lacks optimized XLA 4-bit unpacking kernels).
- If running **benchmarks, batch evaluations, or offline generation**: Run **W8A8** with **INT8 KV Cache** using AQT/MaxText to push the TPU v5e to its full 393 TOPs capacity.
